DevSecOps Pentesting – project pipeline

Blue = automated scanning **Green** = manual pentest **Yellow** = lab and code **Purple** = decision and output

PHASE 1 – DevOps lab
GitLab CE, runner and container registry deployed in Docker

PHASE 2 – Vulnerable repository
VAmPI web API and CI/CD pipeline with planted weaknesses: dummy secrets, excessive permissions, vulnerable dependencies

AUTOMATED TRACK
what the tools detect

PHASE 3 – Secrets audit
Gitleaks scans the repository and its Git history
→ `gitleaks.json`

PHASE 4 – SAST
Semgrep analyses the application code
→ `semgrep.json`

PHASE 5 – Dependencies and images
Trivy: requirements, Dockerfile and built image
→ `trivy-fs.json`, `trivy-image.json`

MANUAL TRACK
what an attacker can do

PHASE 6 – Accounts and tokens
Review of user permissions, CI/CD tokens and authentication mechanisms

PHASE 7 – Pipeline abuse scenarios
Controlled simulations of pipeline compromise, with screenshots, traces and evidence

PHASE 8 – Aggregation and ATT&CK mapping
One JSON file per tool merged and every finding mapped to a MITRE ATT&CK technique and severity

PHASE 9 – Combined report
LaTeX PDF with all results in ATT&CK format, plus Navigator layer

PHASE 10 – Hardening
Secrets removed and rotated, dependencies updated, non-root image, least-privilege tokens

PHASE 11 – Security gate and comparison
Controls integrated in the pipeline, re-scan, security before and after hardening

PHASE 12 – Deliverables
LaTeX report, technical documentation, code and configurations, slides, 2-minute demo video

re-run the scanners on the hardened version